

Deep learning - assignment 1 report

By Guy Shindel (211306386) and Yuval Rays (323041145)

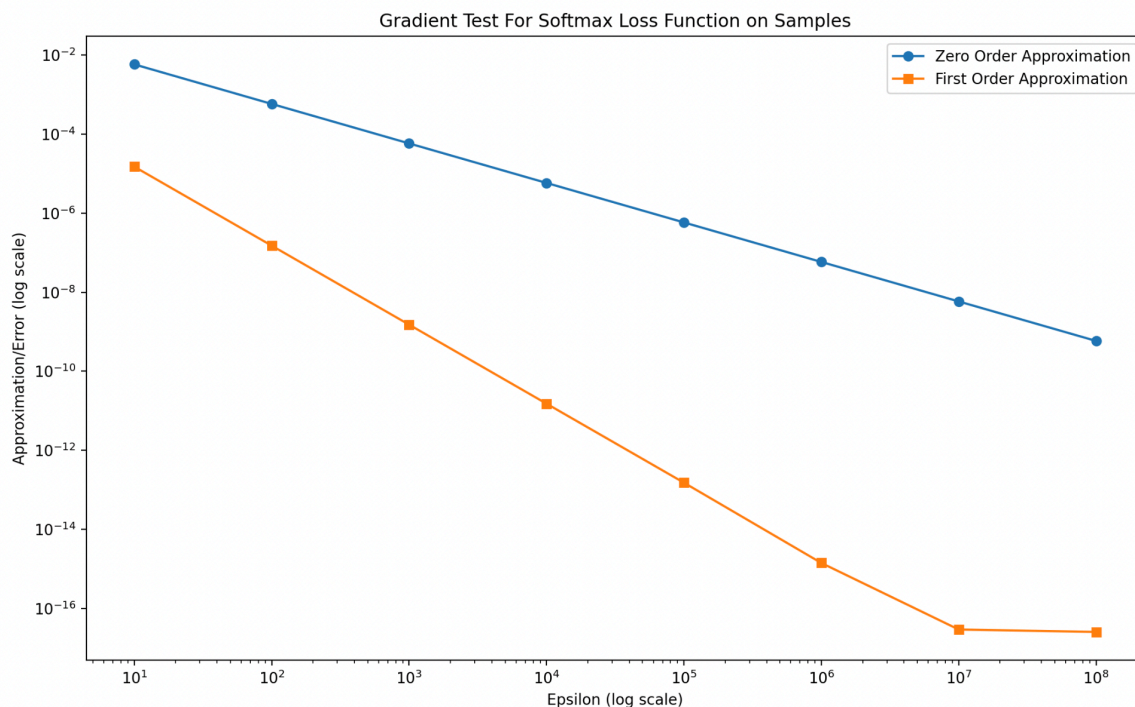
Section I:

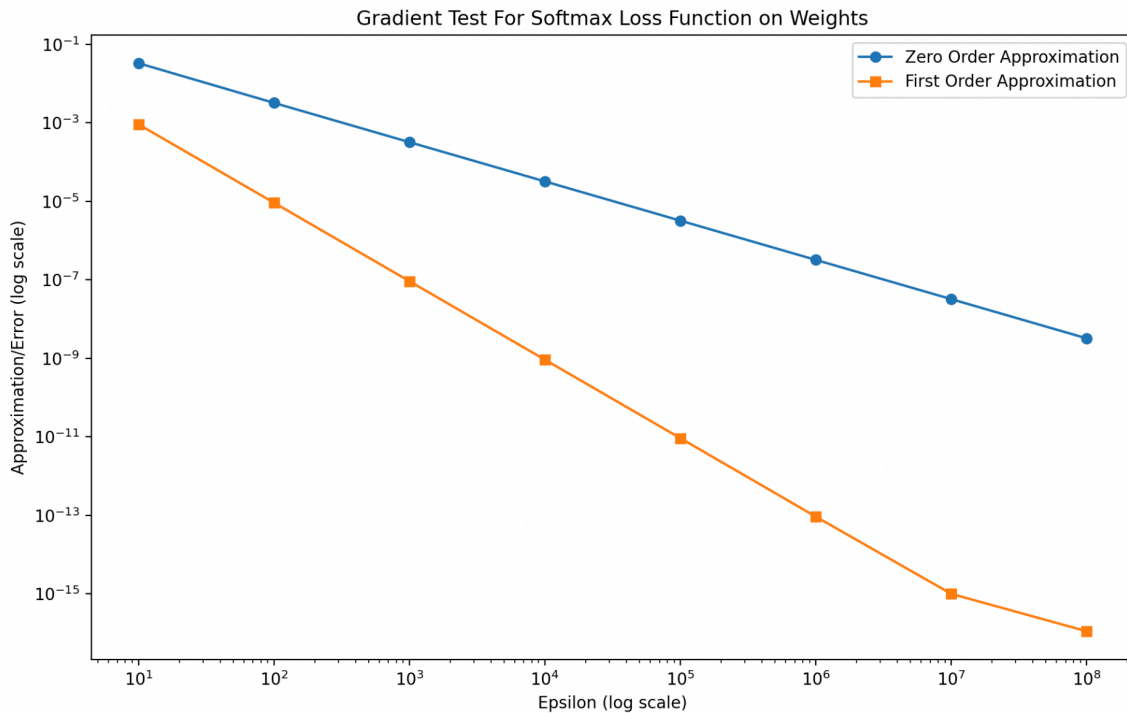
In this section we go over the implementation and results of various tests on the output layer and the Stochastic Gradient Descent (SGD) algorithm. We will mainly focus on the softmax function and ensure it works properly with gradient tests, and presenting the SGD minimizing results for both mean squared error function and our softmax function.

1.1 Softmax implementation:

At this part we had built an implementation of the softmax loss function, and it's various gradient functions (both on samples and weights). Here we represent the results of the gradient tests for both gradient functions.

We have calculated the gradients analytically on the same d vector on multiple epsilons to show the results.

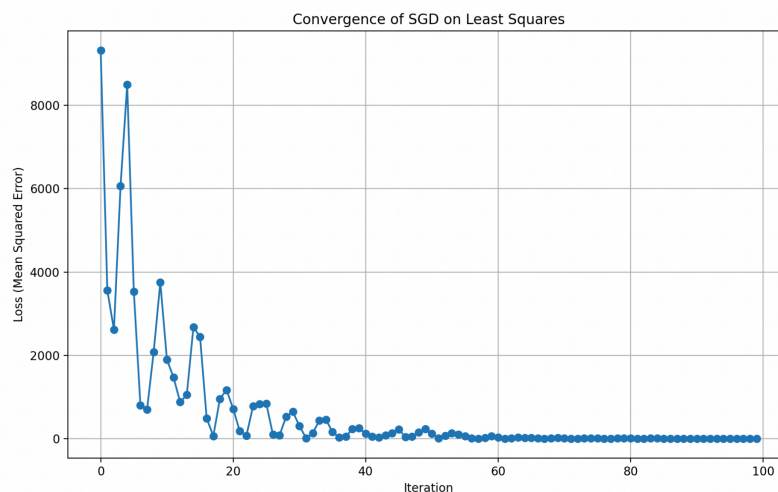




As we can see, our results follow a pattern. On a logarithmic scale, the slope of the first order approximation of the gradient is twice as large as the zero order approximation. This result indicates that the zero order approximation follows an $O(\epsilon)$ slope while the first order approximation follows a $O(\epsilon^2)$ slope. This is expected and shows the gradient is indeed correct.

1.2 SGD implementation:

In this section we implemented a SGD implementation with momentum. In order to test our SGD we created a simple mean loss squared function with its gradient of loss, and we sought to minimize said loss function. Our tests were on a small completely random set of data (100 samples).

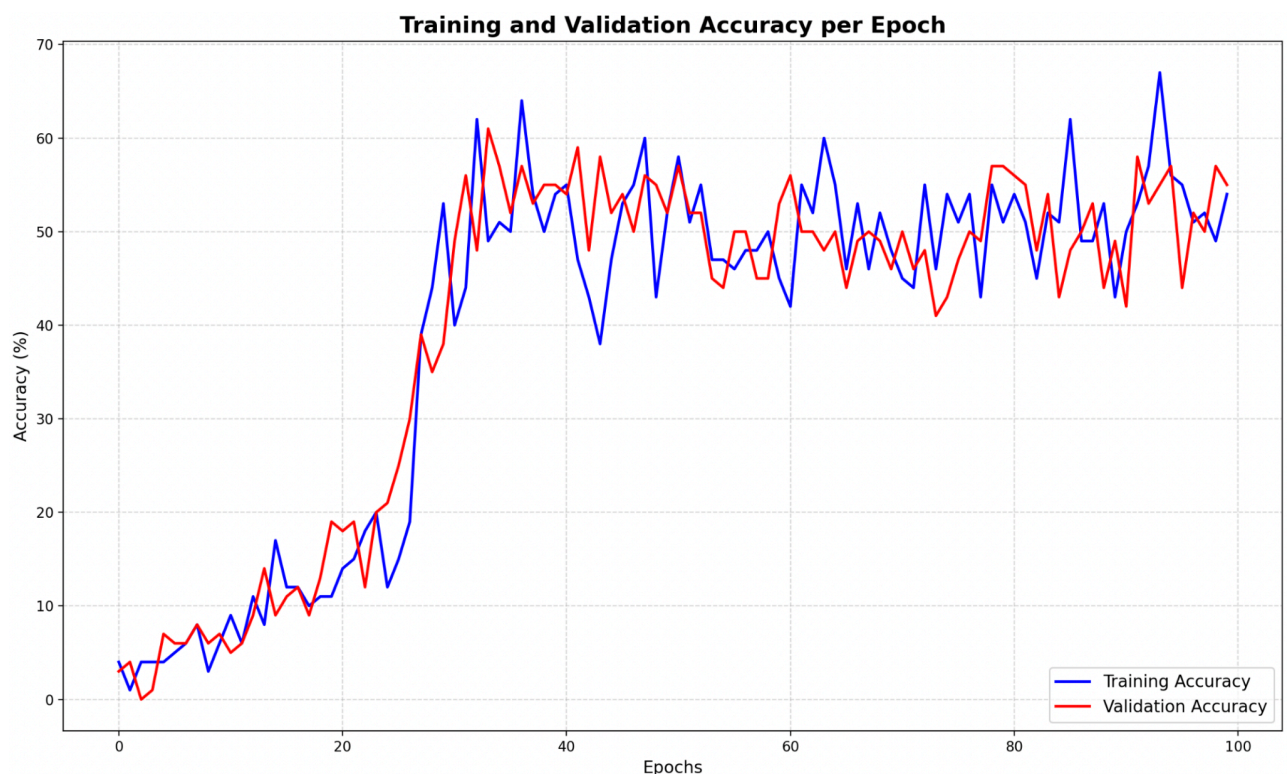


As we can see, the total loss function lowers with more iterations, proving the SGD algorithm is working as intended and the loss function is minimized.

It took some tweaking around to find the hyper parameters best suited for this test. We had noticed that lowering the learning rate up until 0.1 achieved better results, and anything lower started to decrease in performance. Meanwhile the results also improved with lowering the batch size up until around 3. Again im reminding this is on a very small batch of only 100 samples

1.3 SGD on softmax:

At this part we had tested the SGD algorithm on our softmax function from the very first section. This time we used the peaks data sets, and added to our SGD an option to calculate the current success rate of the model on a small random subsample of the training and validation data.



As we can see, we improve our results with the algorithm to around 55% percent, which makes sense for such a limited model.

The hyper parameters for these results were 0.001 learning rate and a batch size of 100. Here too, it took a few tries to get these best results, for any larger learning

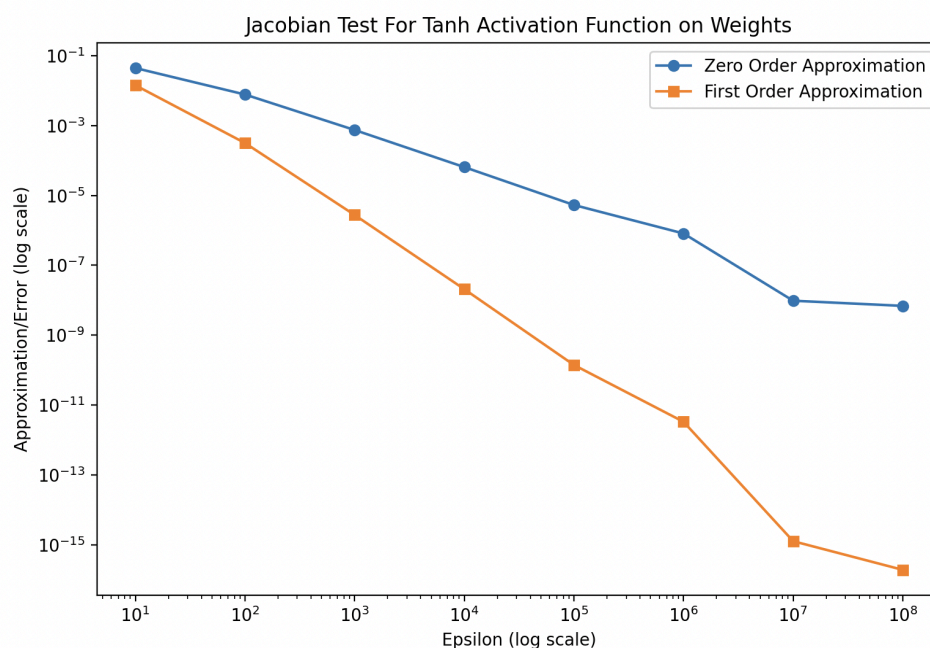
rates the loss function didn't improve and stayed at around 20%, meanwhile lowering the batch sample still resulted in mainly the same results as before at around 50%

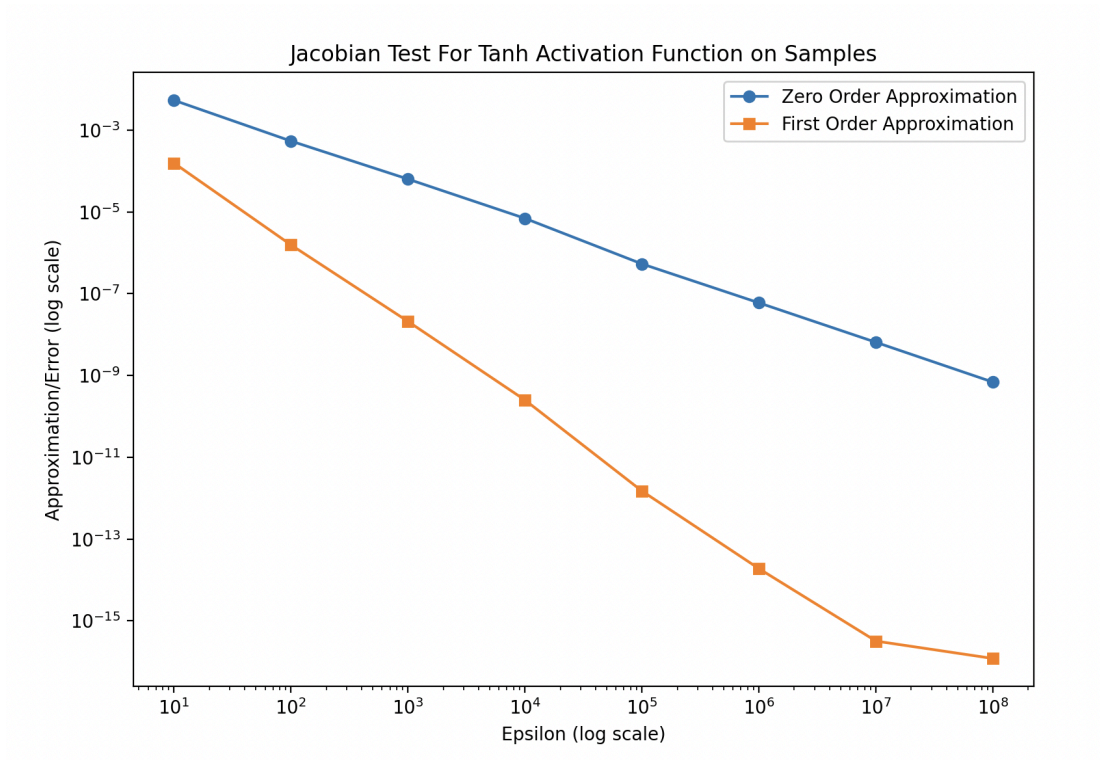
Section II:

At this section, we go over an implementation of the full neural network, both classical and resnet neural networks. Here too we will first conduct tests to analyze whether each one of our jacobians and gradients are indeed correct, and lastly we will train the network with the SGD algorithm we had implemented from section 1.

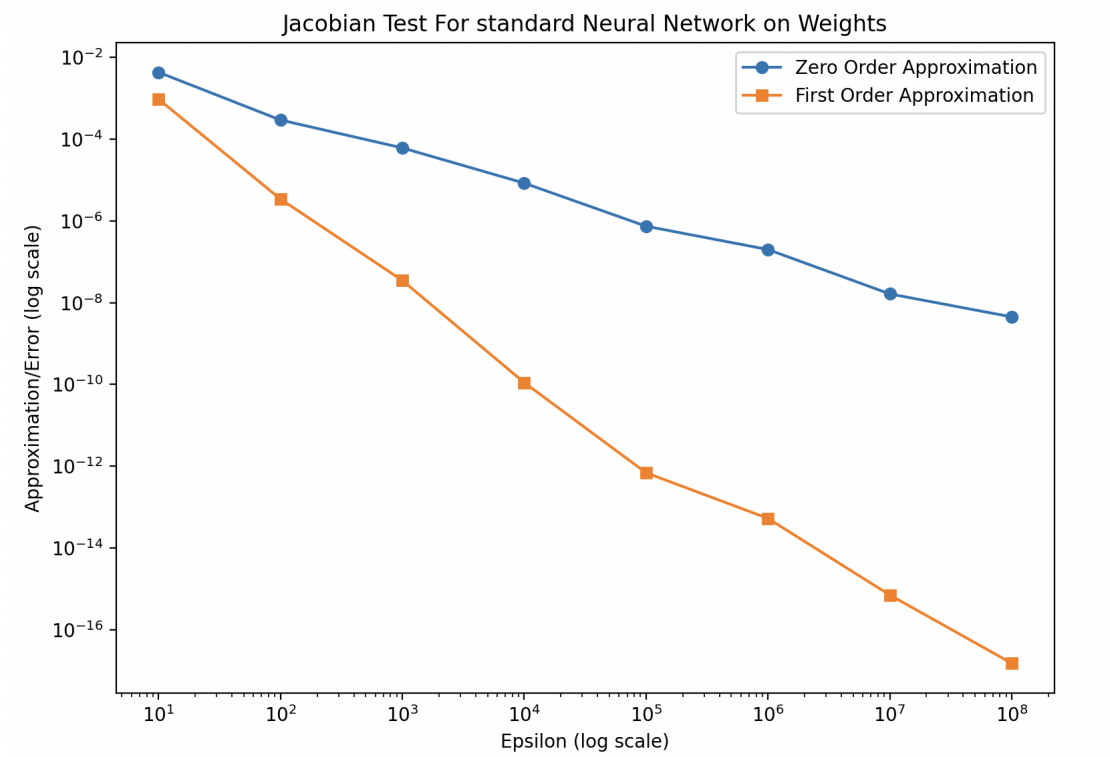
2.1 Jacobian of the network, without an output function

In this section we aim to plot jacobian tests for our classical feed forward neural network. We will first conduct the jacobian tests for each hidden layer, and later do the same for a network with multiple hidden layers (calculated by applying back propagation).





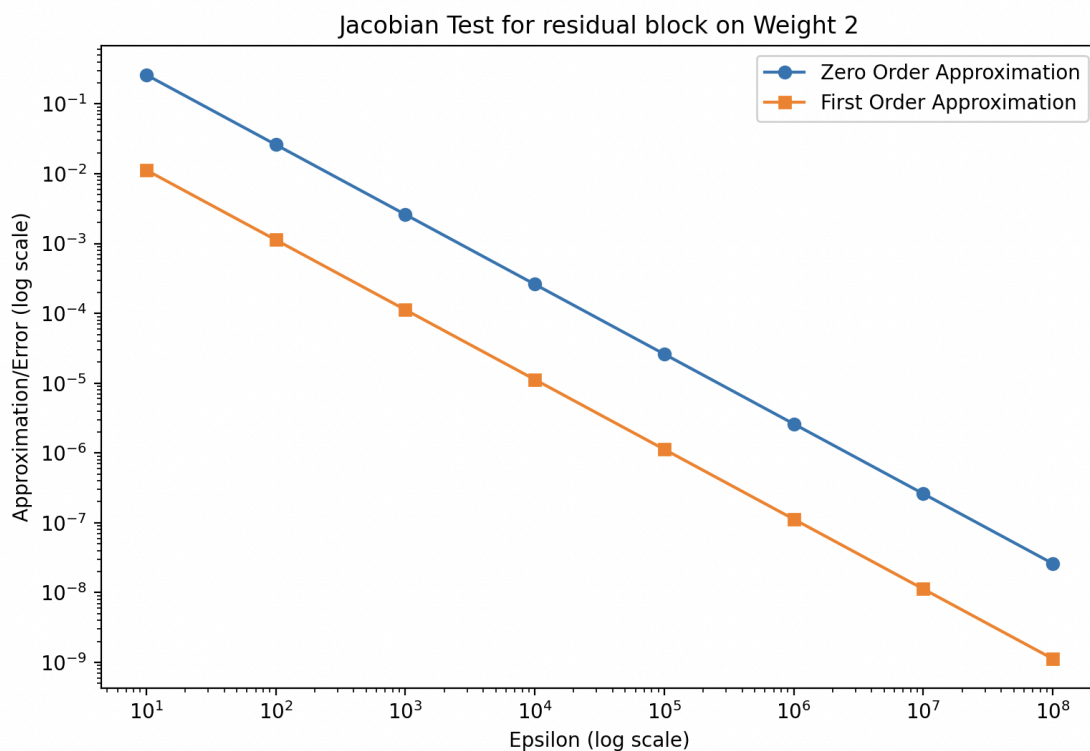
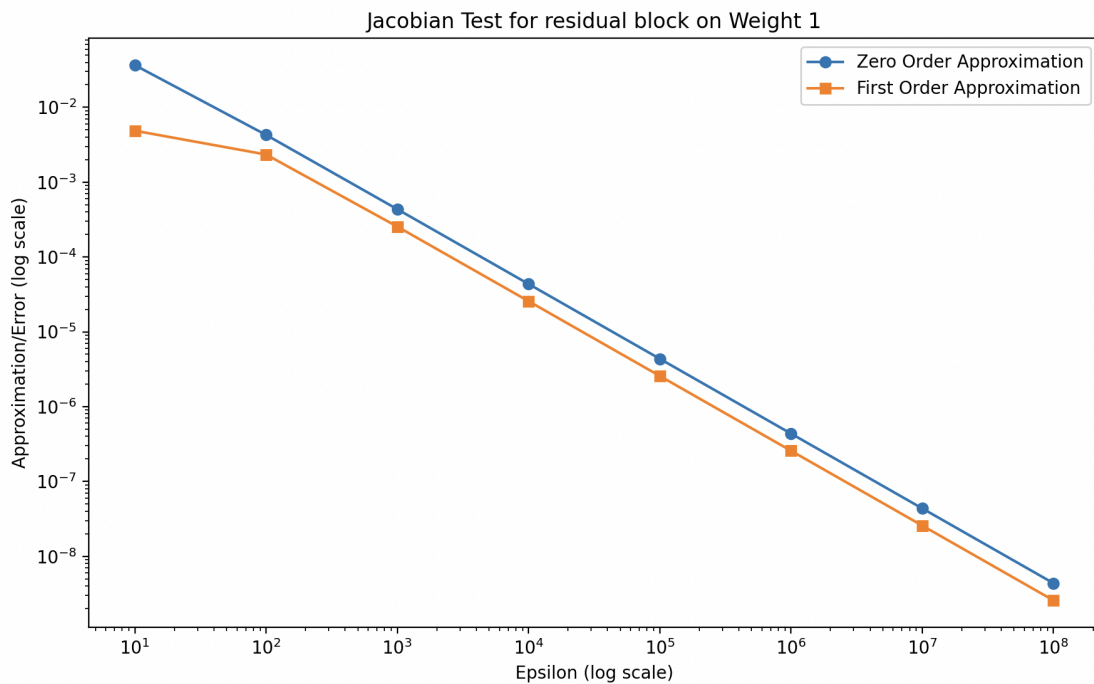
The jacobian test for the entire Neural Network:

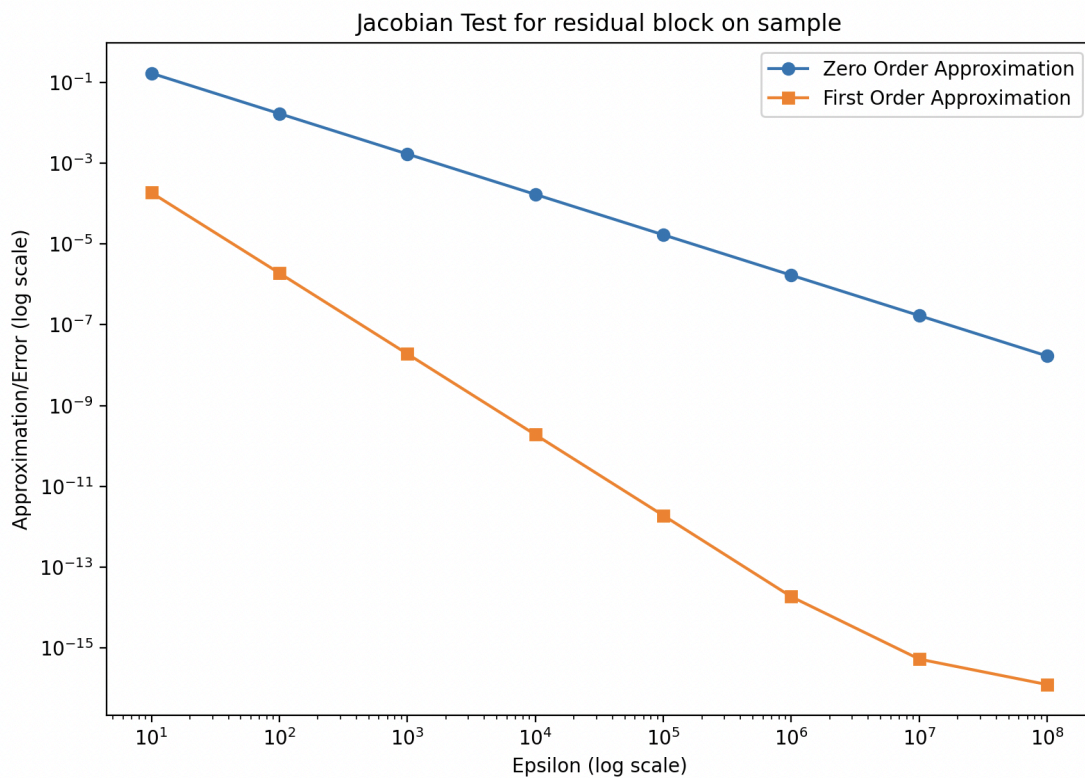


Here too we see a pattern similar to section 1.1, on a logarithmic scale the plotted graphs are following a correct pattern of slopes, which indicates a correct final result.

2.2 Jaciobian test for a resnet neural network:

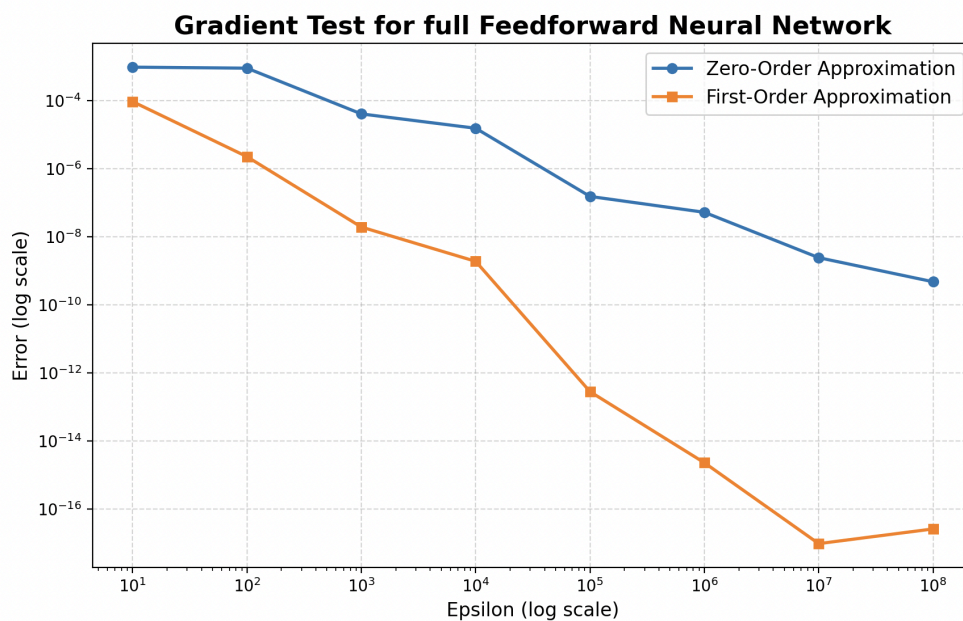
In this section we aim to replicate the previous section, but with the resent neural network. However sadly the results here are not correct as they do not follow the correct plotting behaviour, still we have tried and will add the results we have achieved.





2.3 Combining the network and output function

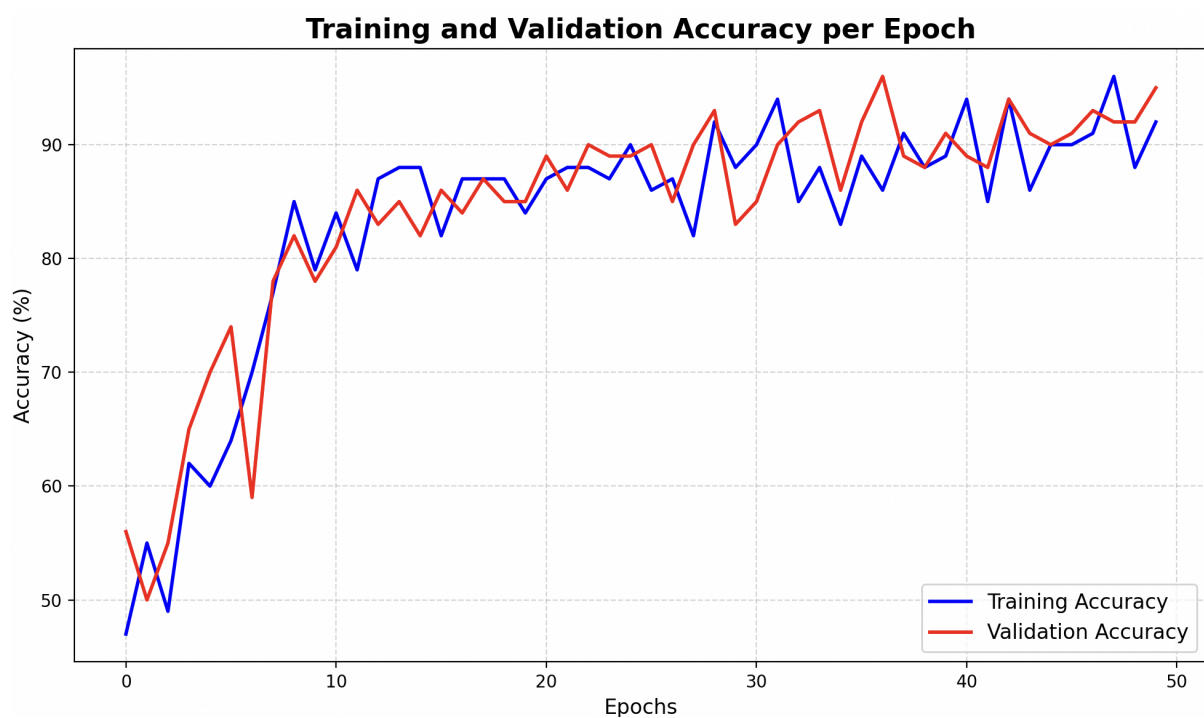
In this section we have combined the network from 2.1 and the softmax output function of 1.1 into a final large model. This model outputs a loss, as such this test is a gradient test on the loss of the model, which is calculated with the back propagation algorithm. Here are the results we had found for the full network



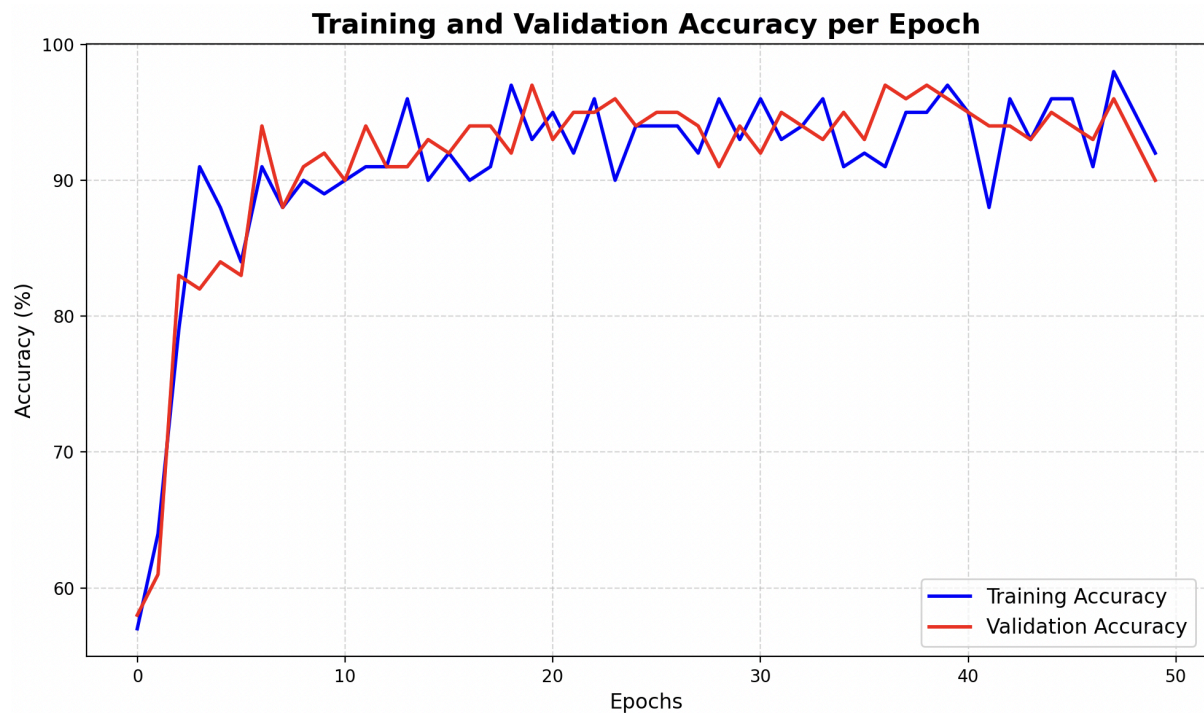
2.4 Training the entire network:

In this section, we used the network from the previous section and trained it on the GMM data set. The test is similar to section 1.3, in which we had calculated after each epoch the success rate of the model on a small random subsample of the training and validation data.

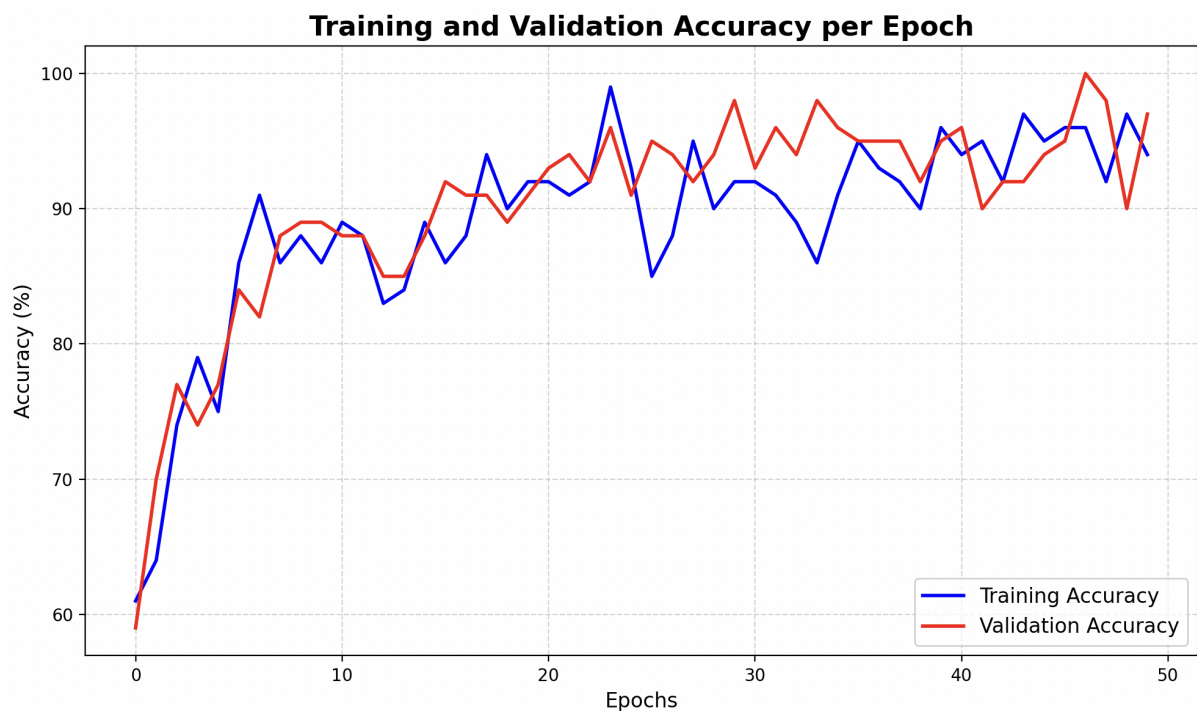
In the graphs that follow we go over multiple network architectures with various layers and sizes in order to show the effects of different models on the final results:



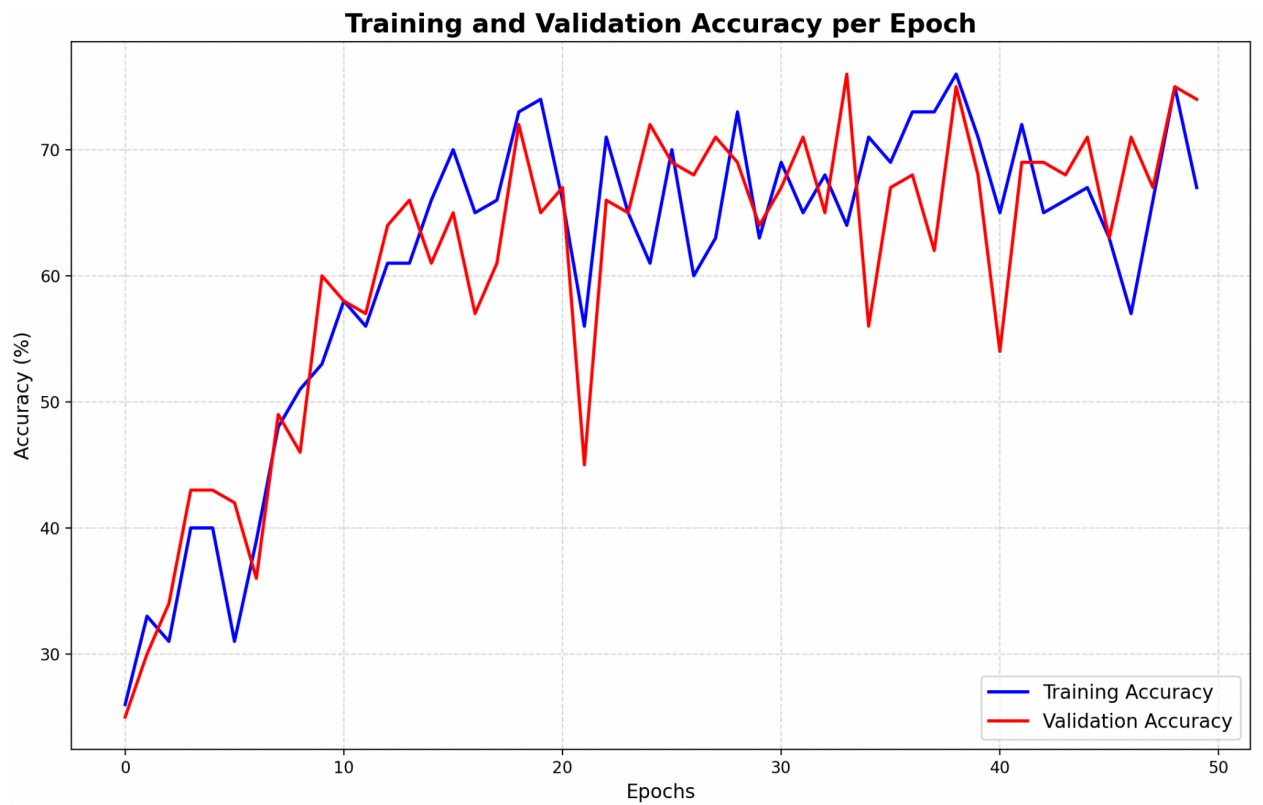
This network is a shallow network layer, with a single hidden layer of 8 neurons. As we can see it takes around 50 epochs to reach over 90%. (0.003 learning rate and 100 batch size).



This network is a deeper network, this time with three hidden layers with each layer having 8 neurons. As we can see we reach over 90% much quicker with around 6 epochs. (0.003 learning rate and 100 batch size).



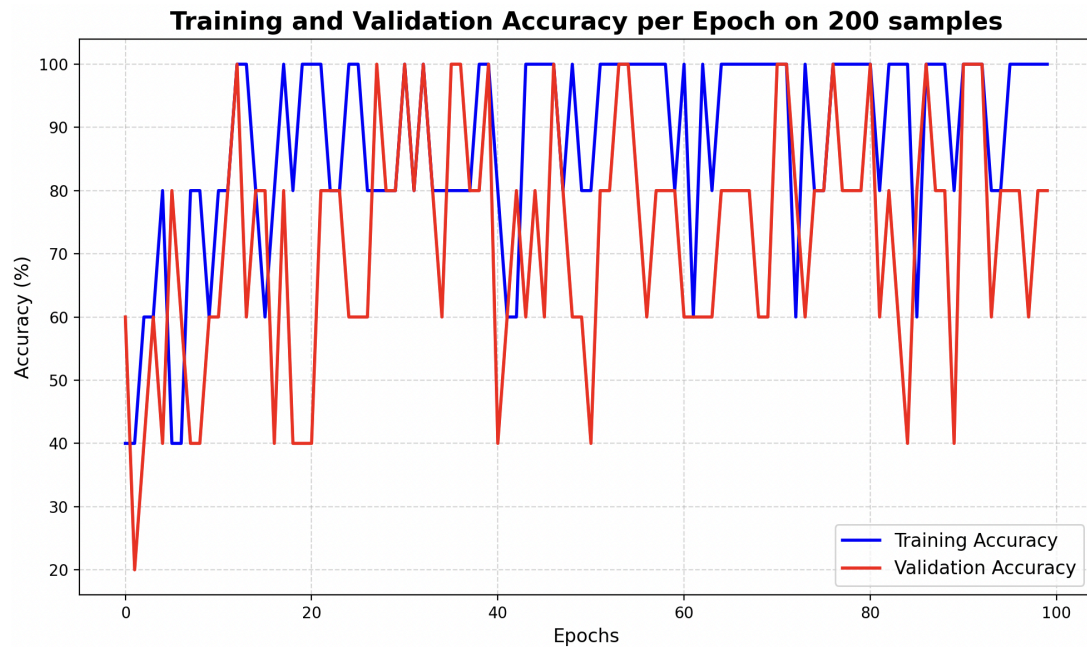
This network is with the same structure of the previous layer, but with a much larger learning rate, 0.01 instead of 0.003. Causing the network to also work slower then before.



This network is a deep network but a smaller one, with 3 hidden layers with 4 neurons each. The hyper parameters are the same (0.003 and 100 batch size), showcasing the need for larger layers.

2.5 Training on a small sample

In this section we aim to show the need for a large training data set to achieve correct results. Here the test is exactly the same as in section 2.4, however, here the model is trained only on random 200 samples from the entire data set.



As we can see, on 200 random samples and the same best case neural network (3 layers of 8 neurons, 0.003 learning rate and 100 batch size) we see a case of overfitting on the small amount of inputs.

Even though the training accuracy is around 100%, the validation accuracy fluctuates to even as low as 40%. Showcasing that the system is trained, but not actually learning the entire scope of the data but only on the scope of the data set.